The MediScan Audit: pneumonia screening for Arogya Diagnostics

ML Practical Assignment, technical report

Mitu Kumar (B24CS029)

Every number in this report is written to a json file in `results/` by the script shown in brackets. `run_all.sh` reproduces all of them; `make_tables.py` collects them into `reports/tables.md`. Framework: PyTorch. Seeds for Python, NumPy and PyTorch are set and logged in every run. The runs reported here were made on a Kaggle T4 GPU. The exact split used is saved as `results/split_used.csv`.

1 Audit of the data (Part 1, tasks 1–3)

Inventory `[audit.py]`. Train 1341/3875, test 234/390, val 8/8 (NORMAL/PNEUMONIA). All 5,856 images are true greyscale (283 are stored as RGB, but with three identical channels).

The classes differ in acquisition, not just anatomy. Mean brightness is identical (122.4 vs 122.9, AUC 0.51), but NORMAL films are much larger: median 1654×1323 vs 1160×776 px. Image height alone separates the classes with ROC-AUC 0.915, area with 0.91 and aspect ratio with 0.84 (Fig. 1). A model trained on these folders can learn "small, wide file ⇒ pneumonia". That rule says nothing about lungs, and it will not hold at a camp that uses a single X-ray machine.

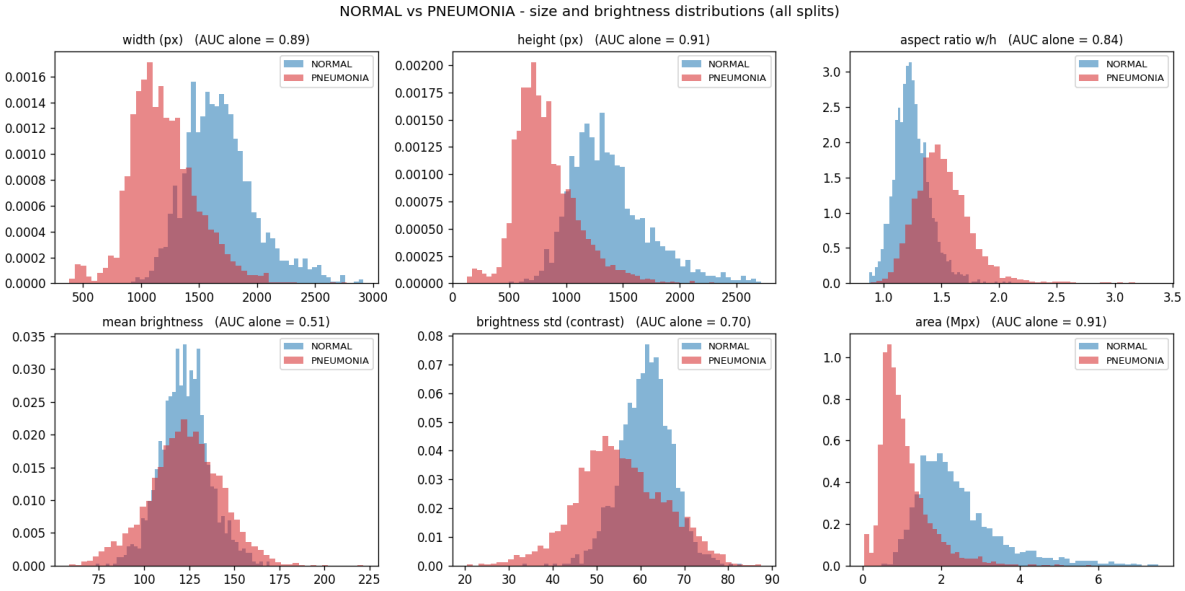


Figure 1: Size and brightness by class. Brightness overlaps; size does not.

Duplicates. 30 exact (md5) duplicate groups, all inside one folder. pHash at ≤4 bits flags 265 pairs (26 train–test), but on inspection those were different children; paediatric AP films look alike at 8×8 DCT. I therefore used pHash (≤10 bits) only for candidates and confirmed them with pixel correlation (NCC≥0.97 at 64×64): **34 confirmed pairs, all within a split, zero across train and test** (highest train–test NCC 0.965, a different patient; `figures/p1_worst_pairs.png`). The same `personNNN` id appears in train and test 170 times. Same-id pairs within train are more alike than random pairs (median pHash 24 vs 26), but across train and test they are not (24 vs 24), so the test folder uses its own numbering. These ids are not evidence of leakage.

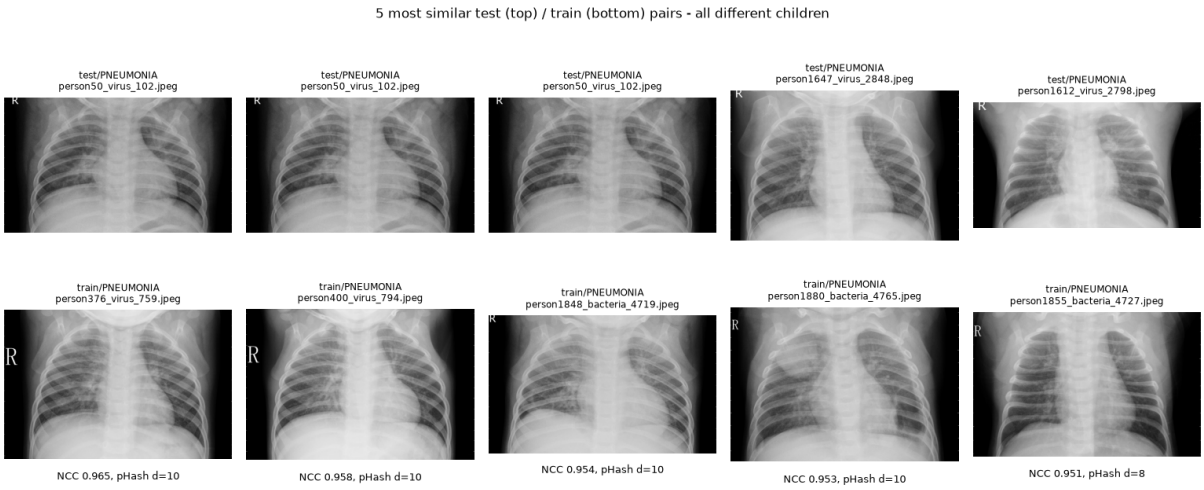


Figure 2: The five most similar test/train pairs `[make_report_figs.py from audit.json]`. Different children: different markers, rib counts and heart outlines. A true duplicate has NCC ≈ 1.0 (`figures/p1_true_duplicate.png`).

Validation set. 16/16 correct gives a 95% Clopper–Pearson interval of $[0.025^{1/16}, 1] = [0.794, 1]$ (Wilson $[0.806, 1]$). **Replacement** `[split.py]`: I pooled all images and grouped them by patient (within a folder) plus confirmed duplicates (3,287 groups). I then made a *locked* stratified group test set (821 images, 227/594) and 5 stratified group folds on the remaining 5,035 (fold 0, 960 images, is the validation set). **Chosen: stratified group split with a locked test set; rejected: stratified k-fold as the replacement**, because every threshold and design choice would then be tuned on the data I report. A single split alone gives no spread. The combination gives both.

2 Shortcuts and baselines (tasks 4–5)

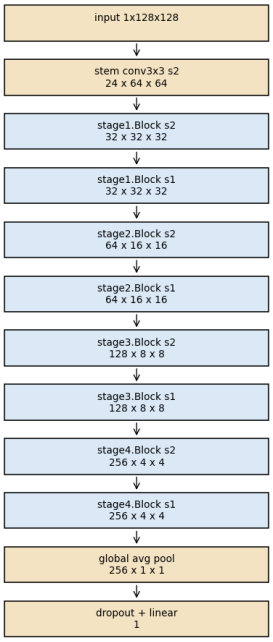
model (fold-0 validation, threshold 0.5)	acc	bal. acc	precision	recall	F1	PR-AUC / ROC-AUC
majority class	0.731	0.500	0.731	1.000	0.845	0.731 / 0.500
LR on 16×16 image	0.941	0.923	0.957	0.962	0.959	0.992 / 0.980
LR on 16×16, **lungs greyed out**	0.894	0.853	0.916	0.942	0.928	0.973 / 0.941
LR on aspect + area only	0.842	0.778	0.874	0.916	0.894	0.958 / 0.905
HOG + logistic regression	0.963	0.949	0.970	0.979	0.974	0.996 / 0.990

Table 1: Crippled models and baselines `[train.py --exp shortcut, baselines]`.

With the lungs removed, the border of a 16×16 image still gives AUC 0.94, and two numbers about the file give AUC 0.91 (chance accuracy is 0.73). Most of what makes this dataset "easy" is available without looking at the lungs, and any CNN trained on it can use these cues. HOG + LR (AUC 0.990) is the bar a deep model has to clear.

3 Deep models (tasks 6–9)

ResNet18 transfer `[--exp transfer]`. Expectation: the low-level ImageNet filters (edges, textures) should transfer to greyscale radiographs, while the high-level ones should not matter. I fed the grey image into all three channels with ImageNet normalisation and fine-tuned the whole network at 224 px. Over 3 seeds: accuracy 0.980 ± 0.002, ROC-AUC 0.996 ± 0.000, PR-AUC 0.999 ± 0.000. This was the best and most stable single model, as expected, but at 11.2M parameters it is a 44.8 MB file, which is over the 25 MB limit.



ArogyaNet `[models.py]`. My own design, trained from scratch, **417,681 parameters**. A stem 3×3/2 conv is followed by 4 stages × 2 blocks (32/64/128/256 channels, first block stride 2). Each block is depthwise-separable conv → depthwise-separable conv → squeeze-and-excitation → + skip → ReLU. Global average pooling replaces flatten+dense, followed by dropout and a linear layer. Input $1 \times 128 \times 128$. The full layer table with output shapes and parameters is in `results/arogyanet_table.md`; the two stage-4 blocks hold 74% of the weights.
Hypotheses (written before the ablations, `reports/hypothesis_arogyanet.md`; it is in the first git commit, made before the Kaggle run that produced the ablation numbers): (1) residual skips give the gradient a short path, expect +0.5–1 AUC point and smaller seed spread; (2) SE re-weights channels by global context, expect +0.3–0.5 points, possibly inside the noise.

fold-0 val, 3 seeds	ROC-AUC	ΔAUC (paired)	bal. acc
full	0.990 ± 0.001	–	0.941 ± 0.011
no residual	0.978 ± 0.003	$\mathbf{-0.0118 \pm 0.0029}$	0.918 ± 0.010
no SE	0.986 ± 0.004	-0.0042 ± 0.0033	0.928 ± 0.007
two-branch (+meta)	0.991 ± 0.001	$+0.0008 \pm 0.0014$	0.951 ± 0.009
96 px	0.991 ± 0.001	$+0.0004 \pm 0.0006$	0.952 ± 0.006
224 px	0.991 ± 0.000	$+0.0008 \pm 0.0011$	0.954 ± 0.003
no augmentation	0.986 ± 0.002	-0.0040 ± 0.0009	0.941 ± 0.002
flip + rot + zoom	0.990 ± 0.001	$+0.0001 \pm 0.0018$	0.944 ± 0.000

Ablation results. *Residual: supported.* Removing the skips costs 1.2 AUC points on every seed, four times the spread, and recall falls from 0.978 to 0.945. The "smaller spread" part of the hypothesis did not hold. *SE: inside the noise.* AUC dropped on all three seeds, but by only 0.0006, 0.0049 and 0.0071. The mean is under two standard deviations, so I cannot claim SE helps. ArogyaNet ties HOG+LR and stays below ResNet18.

Two-branch `[--exp twobranch]`. Adding (mean intensity, aspect, area) through a 3→16→16 dense branch changes nothing measurable ($+0.0008 \pm 0.0014$). This is not because the numbers are uninformative: alone they give AUC 0.91. The image branch already reads the same cues, as task 4 showed. Had the branch helped, the model would have learned something about *how these patients' films were saved*, not about pneumonia. At a camp with one machine that signal would carry no information, so I did not ship this version.

Resolution and augmentation `[--exp res_aug]`. Going from 96 to 224 px makes no difference (both within 0.001 of 128 px), so I kept 128 for speed. Augmentation helps a little (no augmentation: -0.004 ± 0.001). **Horizontal flipping is medically wrong** for chest films. It moves the heart to the right (dextrocardia/situs inversus), mirrors the stomach bubble and the "R" marker, and flips the meaning of any side-specific finding (right-lower-lobe vs left). The flipped image is therefore not a valid example of its label. Here it was harmless to the score, but that only shows the score cannot see the problem. The final model uses ±10° rotation, 0.9–1.1 zoom and small shifts, with no flips.

4 Making it honest (Part 2)

Tuning and regularisation `[part2_tune.py]`. Random search over learning rate ($10^{-3.7}$–$10^{-2.3}$), batch (16/32/64), dropout (0–0.5) and weight decay (10^{-6}–10^{-2}). Budget: 12 configurations × 8 epochs, scored by fold-0 AUC. Best: lr 1.5×10^{-3}, batch 32, dropout 0.22, wd 1.7×10^{-6} (AUC 0.9913). The final model was trained with these settings for 15 epochs, with rotation/zoom augmentation and early stopping on validation AUC (patience 5). Fig. 3: without dropout, weight decay or augmentation, **overfitting begins at epoch 9**. Validation loss bottoms out at 0.177, then climbs to 0.20 while training loss falls to 0.001. The regularised run bottoms out later (epoch 13) and much lower (0.092), and stays flat. Early stopping at epoch 9 would have saved the unregularised model about 14% in validation loss (i.e. calibration). AUC alone barely moved after epoch 9, so early stopping mainly protects the *probabilities*, which matter because the decision is made by thresholding them.

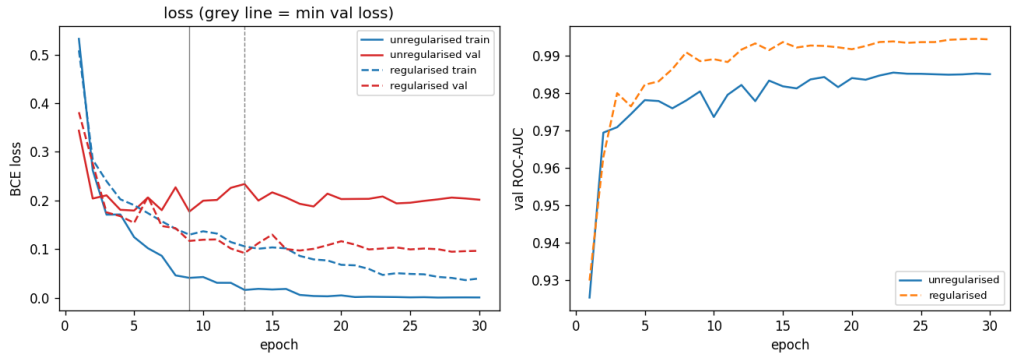


Figure 3: Unregularised vs regularised training (30 epochs, no early stopping). Grey lines: minimum validation loss.

Imbalance and the threshold `[part2_threshold.py]`. Class weights (pos_weight $= n_{\text{neg}}/n_{\text{pos}} = 0.37$) change the *model*: ROC-AUC 0.9916→0.9924, Spearman rank correlation between the two models' scores is only 0.960, so they order patients differently. Threshold moving changes only the *decision*. The ROC curve is the same curve and only the operating point slides along it: moving the plain model to $t = 0.69$ reproduces the weighted model's specificity (0.938), with recall 0.967 vs 0.976. **Cost matrix:** FN = 20, FP = 1, camp prevalence $\pi = 0.1$. Expected cost per child $= 20\pi(1 - \text{TPR}) + (1 - \pi)\,\text{FPR}$, minimised on validation at $\mathbf{t^* = 0.448}$ (0.106 per child; 0.114 at $t = 0.5$; 2.0 for "always normal", 0.9 for "refer everyone"). Fig. 4 marks t^* on the ROC curve and on the PR curve re-weighted to 10% prevalence, where precision is only 0.55. **Refer band**: $[0.109, 0.899]$, from the 99.5%-sensitivity point to the 98%-specificity point on validation.

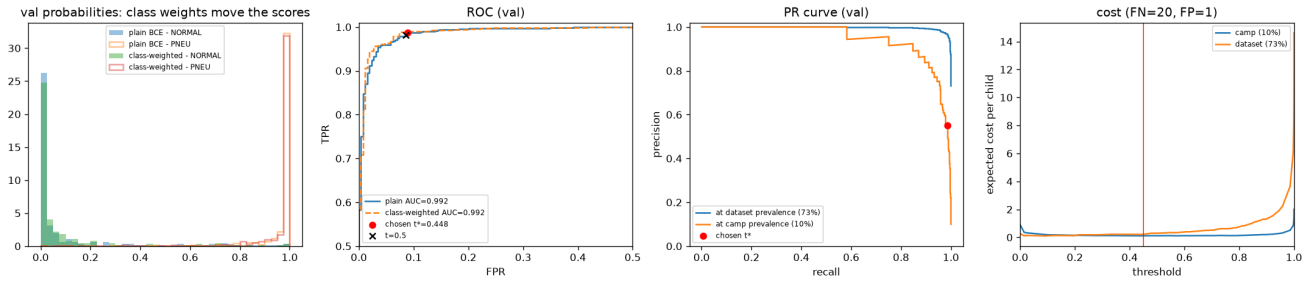


Figure 4: Validation probabilities, ROC, PR at dataset and camp prevalence, and expected cost vs threshold. Red = chosen t^*.

Evaluation `[part2_cv.py, evaluate_test.py]`. 5-fold patient-grouped CV on the pool at t^*: accuracy 0.959 ± 0.007, balanced accuracy 0.947 ± 0.010, precision 0.971 ± 0.009, recall 0.974 ± 0.010, specificity 0.920 ± 0.025, F1 0.972 ± 0.005, ROC-AUC 0.990 ± 0.003, PR-AUC 0.996 ± 0.002. **Locked test, run once** (821 images): TN 205, FP 22, FN 13, TP 581, i.e. accuracy 0.957, precision 0.964, **recall 0.978**, specificity 0.903, F1 0.971, ROC-AUC 0.988, PR-AUC 0.995. These agree with CV. Of the test images, 11.6% fall in the refer band; on the rest, recall is 0.994 (3 misses) and specificity 0.953. *Why accuracy is the wrong headline:* the test set is 72% pneumonia and the camp is 10%, so the same model has different accuracy at each and a constant "normal" answer already gets 90% at the camp. Accuracy weighs a missed sick child the same as a false alarm, while here a miss costs about 20 times more.

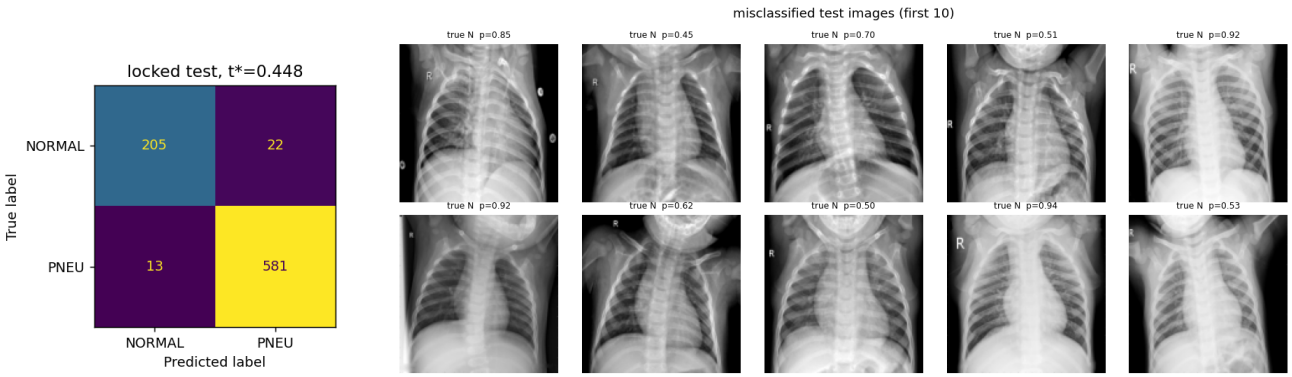
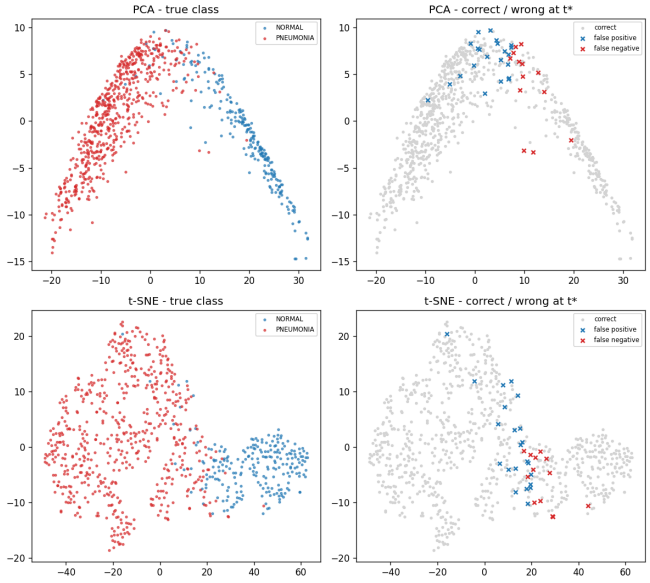


Figure 5: Locked test set at t^*: confusion matrix, and the first ten misclassified images with their probabilities.

Ensembles (locked test, $t = 0.5$, mean over seeds for single models):

model	ROC-AUC	PR-AUC	acc	bal. acc	F1
ArogyaNet full (3 seeds)	0.988	0.995	0.953	0.936	0.968
ArogyaNet no residual / no SE	0.972 / 0.981	0.989 / 0.992	0.924 / 0.944	0.906 / 0.925	0.947 / 0.962
ArogyaNet two-branch	0.991	0.996	0.961	0.946	0.974
ResNet18	0.996	0.998	0.978	0.972	0.985
HOG + LR	0.990	0.996	0.961	0.944	0.973
soft vote: ArogyaNet×3 + ResNet18×3	0.996	0.999	0.978	0.967	0.985
soft vote + HOG/LR	0.996	0.999	0.979	0.968	0.986
Random forest / AdaBoost on ArogyaNet embeddings	0.987 / 0.988	0.992 / 0.995	0.961 / 0.959	0.944 / 0.941	0.973 / 0.972
shipped ArogyaNet	0.988	0.995	0.957	0.945	0.971

The soft vote does **not** beat its best member (ResNet18: same AUC 0.996, lower balanced accuracy 0.967 vs 0.972). The members make highly correlated mistakes: they see the same images and the same shortcuts, and averaging in weaker, similar models pulls the strongest one towards them. RF and AdaBoost on the 256-d embedding equal the CNN's own linear head, so the embedding is already linearly separable and trees add nothing. None of the ensembles fit the 25 MB / 200 ms budget better than a single small model.



Feature space `[evaluate_test.py]`. PCA of the test embeddings: the first component explains 69% of the variance and is essentially the class axis. t-SNE shows two clusters. k-means ($k = 2$) recovers the labels with **ARI 0.825, purity 0.956**. The 35 errors (22 FP, 13 FN) all sit on the boundary between the clusters. **What they share is size**: false positives are NORMAL films that are *smaller* than usual for normals (mean area 1.92 MP vs 2.52 MP for all normals), and false negatives are pneumonia films that are *larger* than usual for pneumonia (1.53 MP vs 1.02 MP). Brightness does not differ (120–122). The final model makes the most mistakes exactly where the acquisition shortcut points the wrong way, so it still partly relies on it.

5 Generalisation test, run once (task 6)

`[external_eval.py]`. COVID-19 Radiography Database, Normal + Viral Pneumonia only, evaluated once at the frozen t^*. The brief says ∼4,600 PNGs; the current release of the database has 10,192 Normal + 1,345 Viral Pneumonia = 11,537, and all of them were used. **The overlap check found that all 1,345 viral-pneumonia images and 1,341 of the normals are copies of our own Kaggle data** (same pHash+NCC rule). Those cannot be called external. The truly external part is 8,851 *normal* films, so only specificity can be measured on it:

Splitting the one external run by where the images came from (counts from `results/p2_external.json`, nothing re-run):

	Kaggle copies, normal (1,341)	Kaggle copies, pneumonia (1,345)	truly external normals (8,851)
internal locked test (reference)	spec 0.903	recall 0.978	–
final model	spec **0.965**	recall **0.967**	spec **0.628**
CLAHE-trained model (the fix)	spec 0.981	recall 0.936	spec 0.540

On all 11,537 together: final model bal. acc 0.820, ROC-AUC 0.935; CLAHE model 0.767 / 0.825. These mix the two very different parts above, so I do not use them as the headline.

How far it falls: on truly external normals, specificity falls from 0.90 to **0.63**, so 37% of healthy external patients would be flagged. That is the pilot's "7 out of 10 healthy children" in smaller form.

Why, and what it is not. My first guess was the Part 1 size shortcut, since every external file is 299 × 299 (aspect 1.0, 0.09 MP), far from our NORMAL class (2.2 MP, aspect 1.23). *The breakdown rules that out*: our own children's films, re-saved by the database authors at 299 × 299 PNG, are still classified at 0.965 specificity and 0.967 recall, as well as on our internal test. So file size, format and the preprocessing pipeline are not the cause. The 8,851 truly external normals come, according to the database's documentation, from the RSNA Pneumonia Detection Challenge (NIH ChestX-ray14), which are mostly **adult** chest films. The drop is a **population shift**: children aged 1–5 versus adults (bigger thorax, different heart-to-chest ratio, denser bone). The pixel statistics

agree. The external films are slightly brighter (mean 129 vs 122) with a second bright mode near 165, and their contrast is more uniform: same mean as our normals (61.6 vs 61.3), much narrower spread (Fig. 6). In other words, the "other hospital" here is mostly a different kind of patient, not just a different scanner.

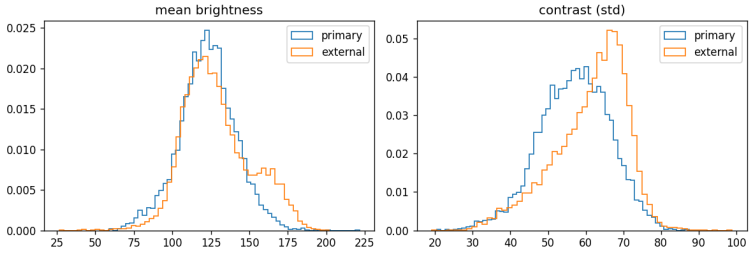


Figure 6: Brightness and contrast, primary vs external `[external_eval.py]`.

The fix (CLAHE on train and test, retrained, threshold re-chosen on validation) **did not help where it mattered**: on the truly external normals, specificity fell from 0.63 to 0.54. It slightly helped on the re-saved Kaggle normals (0.965 → 0.981) but lost recall (0.967 → 0.936). Contrast equalisation cannot turn an adult chest into a child's. An honest fix needs labelled images from the target population and machine, and the tool must not be used on adults.

6 Shipping (task 7) and the report to the CEO

`[predict.py, app.py, ship_check.py]`. `final.pt` is **1.77 MB** (ResNet18: 44.8 MB). On the i7-12700H laptop CPU with 1 thread, one image takes **27.5 ms on average, 59 ms at the 95th percentile**, plus a one-off model load of 19 ms. Both are well inside 200 ms and 25 MB. `predict.py` prints the class and probability. Its checks run in this order: reject colour images; reject blank images (grey std < 8); reject anything whose embedding is further from the training data than the 99.5th percentile of validation (Mahalanobis, Ledoit–Wolf covariance, cut 20.4); return "low confidence – refer to a clinician" inside $[0.109, 0.899]$; otherwise give NORMAL / PNEUMONIA at t^*. **Dog photo**: rejected as colour. **Dog photo in grey**: rejected as out of distribution (distance 34.9 vs cut 20.4), even though its raw probability (0.014) would have said NORMAL. **Black and white images**: rejected as blank. The Gradio demo (`app.py`) wraps the same function and runs offline.

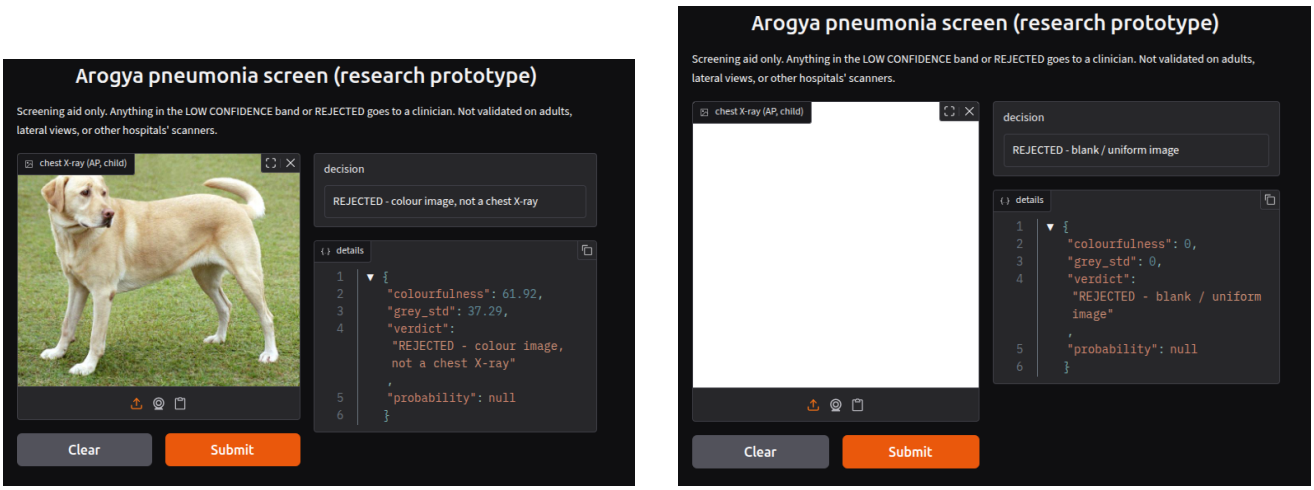


Figure 7: The Gradio demo (`app.py`) on a dog photo and a blank white image: both rejected, no pneumonia/normal answer and no probability.

What the 98.4% really was. I tested an image-level random split (same child in train and test) against a patient-level split `[--exp leak]`: HOG/LR 0.961 vs 0.961, ArogyaNet 0.940 vs 0.952. Leakage does not inflate scores on this data, and ResNet18 reaches 0.980 on my clean split, so 98.4% is a believable *same-hospital* number. It was not a measure of camp performance: same-site tests reward the acquisition shortcut (84% from file size alone), the 16-image validation set could not detect anything, and ResNet50 (≈98 MB) never fitted the channel.

Why "99% accuracy" is the wrong target – arithmetic at 10% prevalence, per 1,000 children. "Always normal": 90.0% accuracy, 100 sick children missed. A "99%" model with sensitivity 0.90 and specificity 1.00: 99.0% accuracy, **10 missed**, expected cost 0.20 per child. Our model at t^* (test sensitivity 0.978, specificity 0.903): **91.1% accuracy, 2 missed**, 87 false alarms, PPV 0.53, NPV 0.997, expected cost 0.13. The model with the lowest accuracy is the safest one. **What to ask for instead**: sensitivity $\geq$0.97 with specificity $\geq$0.85,

measured on 200–300 labelled X-rays from the camp's own machine, under the same 200 ms / 25 MB limits. **It must not be used** as a diagnosis, on a new site or patient population before local validation (outside normals got 37% false alarms), on adults, lateral views or other diseases, or to overrule a clinician. The full CEO report is `reports/CEO_report.md`.

Limitations. 3 seeds is a small sample for ablations of <0.5 AUC points. The external set had no usable pneumonia images. All numbers come from one source hospital. Kaggle's preinstalled package versions shuffled the group folds differently from my laptop; the split actually used is saved in `results/split_used.csv`. GPU training with `cudnn.benchmark` is not bit-for-bit deterministic: the CV fold-0 model and the final model share data, settings and seed, yet miss 28 vs 9 pneumonia cases at t^* on that fold. This is why every comparison here uses 3 seeds or 5 folds, and why the CV spread is the honest error bar.